

Самоорганизующиеся структуры данных

1 Введение

В данном конспекте рассматриваются самоорганизующиеся структуры данных (то есть те, которые могут менять свою структуру по некому правилу/алгоритму), а именно *unsorted linear lists* и их применение для задачи поддержания множества элементов. Пусть у нас есть список из элементов, где каждый элемент содержит указатель на следующий и предыдущий. Чтобы найти элемент, необходимо сделать линейный проход по списку, начиная с головы. Множество элементов списка обозначим S . В процессе обработки запросов алгоритм может динамически изменять порядок элементов в списке, чтобы оптимизировать время будущих обращений.

2 Постановка задачи

Рассмотрим последовательность запросов. Возможны следующие операции:

- **Access(x).** Поиск элемента $x \in S$. Если x находится на k -й позиции в списке, алгоритм проходит k элементов, поэтому затраты на запрос равны k .
- **Insert(x).** Вставка элемента x . Алгоритм сканирует весь список (убеждаясь, что x нет), а затем вставляет x в конец. Затраты равны $n + 1$, где n — текущая длина списка.
- **Delete(x).** Удаление элемента x . Затраты равны k .

Не умаляя общности, далее в анализе мы будем считать, что последовательность запросов состоит только из запросов типа **Access**. Все утверждения легко обобщаются и на **Insert**, и на **Delete**.

Модель стоимостей. Сразу после доступа к элементу x , алгоритм может бесплатно переместить его на любую позицию ближе к началу списка (*free exchanges*). Любой другой обмен двух соседних элементов в списке стоит 1 (*paid exchanges*).

Мы оцениваем эффективность нашего онлайн-алгоритма A , сравнивая его с оптимальным офлайн-алгоритмом OPT , который заранее знает всю последовательность запросов σ . Пусть $C_A(\sigma)$ — суммарные затраты алгоритма A , а $C_{OPT}(\sigma)$ — затраты оптимального алгоритма. Теперь было бы неплохо для любого онлайн алгоритма придумать число, позволяющее сравнивать качество алгоритмов.

Определение: Детерминированный онлайн-алгоритм называется c -конкурентным (c -competitive), если существует константа a , такая что для любой последовательности запросов σ выполняется:

$$C_A(\sigma) \leq c \cdot C_{OPT}(\sigma) + a$$

3 Детерминированные алгоритмы

Рассмотрим три классических детерминированных онлайн-алгоритма:

- **Move-to-front (MTF):** после обращения к элементу он всегда перемещается в самое начало списка.
- **Transpose:** запрошенный элемент меняется местами с непосредственно предшествующим ему элементом.
- **Frequency-count:** ведется подсчет частоты обращений к каждому элементу. Список всегда поддерживается отсортированным по невозрастанию частоты.

Утверждение 1. Алгоритмы *Transpose* и *Frequency-Count* не являются c -конкурентными ни для какой константы c (без доказательства).

Утверждение 2. Для любого детерминированного c -конкурентного алгоритма $c \geq 2$ (без доказательства).

Теорема 1. Алгоритм *Move-to-front* является 2-конкурентным.

Доказательство. Рассмотрим последовательность запросов $\sigma = \sigma(1)\sigma(2) \dots \sigma(m)$. На каждом шаге t будем сравнивать списки, полученные алгоритмами MTF и OPT. Будем называть *инверсией* пару элементов x, y , расположенных в разном порядке в списках MTF и OPT (например, в OPT x идет перед y , а в MTF — наоборот). Введем функцию потенциала $\Phi(t)$, равную количеству инверсий после обслуживания запроса t .

Пусть в момент t запрошен элемент x . Пусть k — число элементов, стоящих строго перед x в **обоих** списках, а l — число элементов, предшествующих x в списке MTF, но находящихся после x в списке OPT.

Тогда фактические затраты алгоритмов на этом шаге:

$$C_{MTF}(t) = k + l + 1, \quad C_{OPT}(t) \geq k + 1$$

Оценим амортизированную стоимость шага для MTF: $a(t) = C_{MTF}(t) + \Phi(t) - \Phi(t-1)$. Переместив x в начало списка, MTF уничтожит ровно l инверсий (с элементами, которые в OPT стоят после x) и создаст не более k новых инверсий. Следовательно, изменение потенциала за счет действий MTF: $\Delta\Phi \leq k - l$. Заметим также, что любой платный обмен (paid exchange), сделанный алгоритмом OPT, стоит ему 1, но может увеличить Φ не более чем на 1, что сохраняет итоговое неравенство.

Тогда:

$$C_{MTF}(t) + \Phi(t) - \Phi(t-1) \leq (k + l + 1) + (k - l) = 2k + 1 \leq 2C_{OPT}(t) - 1$$

Суммируя это неравенство по всем t от 1 до m , и учитывая, что $\Phi(m) \geq 0$ и $\Phi(0) = 0$ (начальные списки совпадают), получаем $C_{MTF}(\sigma) \leq 2C_{OPT}(\sigma)$, что доказывает 2-конкурентность. $\square$

4 Вероятностные алгоритмы

Определение: Вероятностный онлайн алгоритм A называется c -конкурентным, если существует константа a , что для любой последовательности операций σ :

$$\mathbb{E}[C_A(\sigma)] \leq c \cdot C_{OPT}(\sigma) + a.$$

По сути то же определение, что и раньше, но мы усредняем стоимость по всем возможным вариантам действий алгоритма на заданной последовательности операций.

Утверждение 3. Для любого вероятностного s -конкурентного алгоритма $s \geq 1.5$ (без доказательства).

Алгоритм Timestamp(p). Пусть $p \in [0, 1]$ — параметр алгоритма Timestamp (иногда будем сокращать до TS). Запрос $\sigma(t) = x$ обрабатывается следующим образом:

1. Если элемент x ни разу не запрашивался на интервале $[1, t - 1]$, то его позиция в списке не меняется, и алгоритм завершает шаг.
2. Иначе, пусть $t' \in [1, t - 1]$ — момент **предыдущего** запроса к x .
3. С вероятностью p выполняется **Шаг (a)**: элемент x перемещается в самое начало списка.
4. С вероятностью $1 - p$ выполняется **Шаг (b)**: Алгоритм ищет элемент $v_x(t)$, который является **ближайшим к началу списка** среди всех элементов, **предшествующих** x в текущий момент, и удовлетворяет хотя бы одному из условий:
 - i) он вообще не запрашивался в интервале $[t', t - 1]$;
 - ii) он запрашивался ровно один раз в интервале $[t', t - 1]$, и этот запрос был обработан Шагом (b).

Если такой элемент $v_x(t)$ найден, алгоритм вставляет x непосредственно **перед** $v_x(t)$. Если такого элемента нет, мы считаем $v_x(t) = x$, то есть элемент x просто остается на своем месте.

Примечание: **Timestamp(p)** вынужден использовать информацию о прошлых запросах.

Заметим, что при $p = 1$ алгоритм вырождается в Move-to-front.

Теорема 2. Алгоритм **Timestamp(p)** является s -конкурентным, где $s = \max(2 - p, 1 + p(2 - p))$.

Примечание: При выборе $p = (3 - \sqrt{5})/2$ достигается оптимальное для Timestamp(p) конкурентное отношение, равное золотому сечению $\phi \approx 1.62$, что лучше любого детерминированного алгоритма.

Для доказательства сформулируем вспомогательные леммы.

Лемма 1. Пусть x и y ($x \neq y$) — элементы списка. Если x запрашивается в момент t' , а затем в момент $t > t'$, и y не запрашивается в интервале $[t', t]$, то сразу после обслуживания запроса t , элемент x предшествует y в списке.

Доказательство. Если $\sigma(t)$ обслуживается шагом (a), x идет в начало списка и очевидно предшествует y . Если применяется шаг (b), x вставляется перед $v_x(t)$, а значит, перед всеми элементами, которые не запрашивались на интервале $[t', t]$. Элемент y удовлетворяет этому условию. Поскольку позиция y не может поменяться (пока к нему не обратятся), x будет предшествовать y . $\square$

Лемма 2. Пусть в момент t запрошен x , и $t' < t$ — момент предыдущего запроса к x . Если элемент y ($y \neq x$) удовлетворяет хотя бы одному из условий:

- (a) запрашивался хотя бы дважды на интервале $[t', t - 1]$;
- (b) запрашивался ровно один раз на интервале $[t', t - 1]$, и этот запрос был обработан Шагом (a);

то y предшествует $v_x(t)$ в списке в момент t .

Доказательство. Докажем пункт (а) от противного. Пусть t_0 — самый первый момент, когда утверждение нарушается, t'_0 — предыдущий запрос к x , а $z = v_x(t_0)$. Предположим, что y запрашивался дважды, но не предшествует z . Пусть t_y — момент последнего запроса к y на $[t'_0, t_0 - 1]$. Если $\sigma(t_y)$ обработан шагом (а), y попадает в начало и точно предшествует z . Если шагом (б), вспомним, что по определению $v_x(t)$, элемент z запрашивался максимум 1 раз (и через шаг б). Значит z не может предшествовать $v_y(t_y)$. Тогда y вставляется перед z в момент t_y . Чтобы к моменту t_0 элемент z снова оказался перед y , он должен быть запрошен в какой-то момент $t_z \in [t_y + 1, t_0 - 1]$ и вставлен перед y . Но к моменту t_z элемент y уже запрашивался дважды с момента прошлого запроса к z , что противоречит выбору t_0 как самого первого момента нарушения леммы.

Случай (б) доказывается аналогично. $\square$

5 Схема доказательства Теоремы 2

Введем индикаторную величину $C_{TS}(t, x) = 1$, если в момент t элемент x предшествует запрошенному элементу $\sigma(t)$, и 0 иначе. Затраты алгоритма на всей последовательности можно расписать как:

$$C_{TS}(\sigma) = \sum_{t=1}^m \sum_{x \in S} C_{TS}(t, x)$$

Перегруппируем слагаемые, собирая их не по времени, а по неупорядоченным парам элементов $\{x, y\}$:

$$\mathbb{E}[C_{TS}(\sigma)] = \mathbb{E} \left[\sum_{\{x, y\}} \left(\sum_{t: \sigma(t)=x} C_{TS}(t, y) + \sum_{t: \sigma(t)=y} C_{TS}(t, x) \right) \right]$$

Пусть $\sigma_{\{x, y\}}$ — проекция последовательности запросов только на элементы x и y (то есть запросы берем из σ только к x и y). Пусть $\bar{C}_{OPT}(\sigma_{\{x, y\}})$ — стоимость работы оптимального алгоритма на списке, состоящем только из этих двух элементов. Если расписать аналогично формуле выше, то легко будет увидеть, что:

$$C_{OPT}(\sigma) \geq \sum_{\{x, y\}} \bar{C}_{OPT}(\sigma_{\{x, y\}})$$

Такое разбиение позволяет рассматривать относительный порядок только двух элементов $\{x, y\}$, полностью игнорируя остальные элементы. Нам достаточно доказать, что для любой пары суммарное матожидание затрат Timestamp на этой паре не превосходит $c \cdot \bar{C}_{OPT}(\sigma_{\{x, y\}})$.

Последовательность $\sigma_{\{x, y\}}$ разбивается на фазы P_j . Фаза заканчивается, когда после серии запросов к одному элементу (хотя бы 2 запроса к одному и тому же) происходит запрос к другому. Фазы классифицируются на 4 типа (где $h \geq 2$):

1. $P_j = x^h$ (или y^h)
2. $P_j = xy^h$ (или yx^h)
3. $P_j = (xy)^{h_1} x^{h_2}$ (или $(yx)^{h_1} y^{h_2}$), $h_1 \geq 1$
4. $P_j = (xy)^{h_1} y^{h_2}$ (или $(yx)^{h_1} x^{h_2}$), $h_1 \geq 1$

Используя доказанные Леммы 1 и 2, и вычисляя вероятности перемещений элементов на каждом шаге, можно оценить матожидание затрат Timestamp по сравнению с $\bar{C}_{OPT}$ для каждой фазы в отдельности (без доказательства, здесь на самом деле просто технические выкладки):

- Для типа 1: $\mathbb{E}[C_{TS}(P_j)] \leq (2 - p) \cdot \bar{C}_{OPT}(P_j)$
- Для типа 2: $\mathbb{E}[C_{TS}(P_j)] \leq (1 + p(2 - p)) \cdot \bar{C}_{OPT}(P_j)$
- Для типов 3 и 4: $\mathbb{E}[C_{TS}(P_j)] \leq \max(2 - p, 1 + p(2 - p)) \cdot \bar{C}_{OPT}(P_j)$

Поскольку любая фаза для любой пары элементов ограничивается коэффициентом $c = \max(2 - p, 1 + p(2 - p))$, суммирование этих оценок дает искомую конкурентность для всего алгоритма. Теорема доказана.